

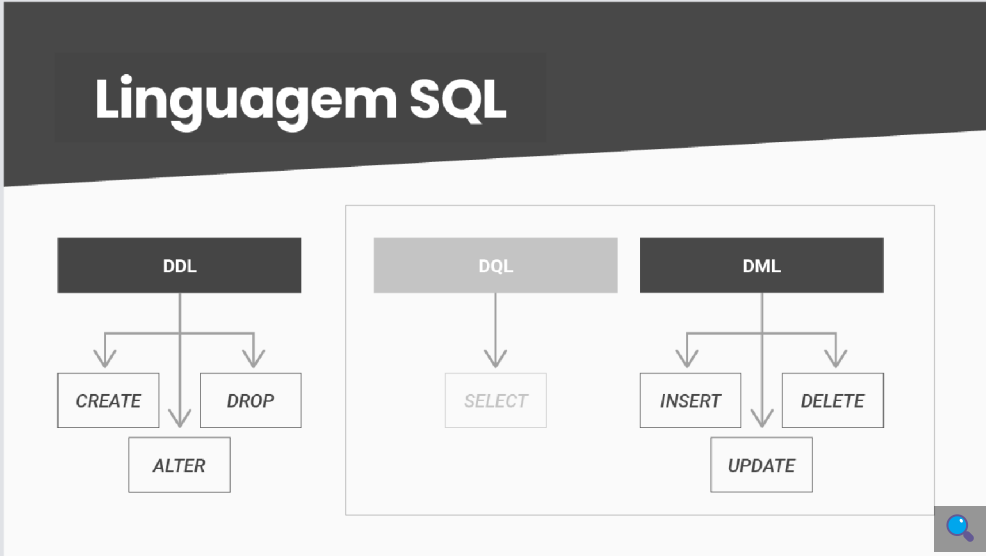


AULA 3

Carga e alteração de dados e criação de novos objetos em esquemas em banco de dados relacionais: atualizando as instâncias e o esquema

Que história é essa?

Anteriormente, foi abordada a conversão do DER em um banco de dados físico por meio da criação de seu esquema com tabelas, atributos e restrições de integridade. Também foram realizadas consultas no dicionário de dados do SGBD. Agora, serão populadas essas tabelas com dados e verificadas as mudanças no estado do banco de dados depois dessas operações. Para isso, serão usados os comandos DML e DQL, destacados a seguir. Além disso, será realizada uma carga de dados em lote utilizando a linha de comando e serão criados outros objetos que também fazem parte do esquema físico de um banco de dados relacional.





Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Atualizar dados do banco de dados com comandos básicos de SQL (INSERT, UPDATE e DELETE).
- Consultar dados do banco de dados com SQL (SELECT básico e sub SELECT).
- Manipular outros objetos do banco de dados (visões, gatilhos e índices).
- Carregar dados externos em um banco de dados.

COMANDOS SQL PARA CRUD DE ENTIDADES

O acrônimo CRUD corresponde às operações **CREATE**, **READ**, **UPDATE** e **DELETE** para entidades. Primeiro, serão vistos os comandos SQL que viabilizam a carga dos dados em um banco relacional. Nesse ponto, volta-se a utilizar a linguagem em sua fração DML (*data manipulation language*), **cujo foco são as instâncias e não mais o esquema**, como é o caso da fração DDL. As instruções que serão exploradas permitem inserir, atualizar e excluir dados nos esquemas relacionais.

DML: INSERT

O comando básico para inserir dados em um esquema relacional é INSERT. A imagem a seguir ilustra duas opções de sintaxe de inserção de dados com a linguagem SQL. Basicamente, é necessário escolher qual tabela deve receber dados e, pela cláusula VALUES, explicitar os valores. A diferença entre a opção (1) e (2) da figura está na especificação da lista de colunas. Quando a lista de colunas é especificada, a lista de valores deve seguir a ordem da lista de colunas do comando. As colunas da tabela cujos valores são opcionais ou gerados automaticamente podem ser omitidas. Quando a lista de colunas **NÃO** é especificada, a lista de valores deve seguir a ordem da lista de colunas da tabela e nenhuma coluna pode ser omitida.

DML

Exemplos da sintaxe	
(1) INSERT INTO nome_da_tabela (coluna1, coluna2, ...) VALUES (valor1, valor2, ...);	(2) INSERT INTO nome_da_tabela VALUES (valor1, valor2, ...);
(3) UPDATE nome_da_tabela SET coluna1 = novo_valor1, coluna2 = novo_valor2, ... WHERE condição	(4) DELETE FROM nome_da_tabela WHERE condição

Na próxima figura, há quatro exemplos de INSERT utilizando a tabela ALUNO do banco de dados da aplicação de EAD que foi criado na última aula. No caso **(a)**, tem-se a lista de colunas especificada. Observe que as colunas MATRICULA e ENDERECO_COMPLEMENTO foram omitidas e que a ordem das colunas especificada é o que deve ser considerado ao definir os valores a serem incluídos. Já no caso

(b), tem-se que a lista de colunas foi omitida. Observe que, na lista de valores, foram especificados oito valores e que o primeiro deles é NULL, ou seja, representa o NULO, e corresponde a MATRICULA. Nesse caso, o banco irá gerar automaticamente o valor da MATRICULA, já que a coluna foi definida com o autoincremento.

Os exemplos (c) e (d) correspondem a comandos INSERT que não foram executados com sucesso. Um caso de violação de integridade em relação aos valores possíveis para a coluna SEXO é exemplificado com o caso (c). Essa coluna permite somente três valores: 'Feminino', 'Masculino' e 'Não informado'. Então, o valor 'Nenhum' foi considerado inválido, gerando um erro, conforme a mensagem em vermelho. No último exemplo (d), tem-se um comando INSERT com um número de valores menor do que o número de colunas da tabela ALUNO. Comparando-se os casos (b) e (d), pode-se perceber que o valor correspondente a MATRICULA não foi especificado, mas a mensagem de erro em vermelho não indica qual seria a coluna faltando.

Note que essa tabela tem duas colunas que não são do tipo texto: DATA_NASCIMENTO, que é NUMERIC (numérico), e ENDERECO_CEP, que é INTEGER (inteiro). Somente é necessário usar as aspas simples (') para delimitar o conteúdo em caso de colunas do tipo texto. Como o SQLite não tem um tipo específico para datas, os valores estão sendo armazenados como números no padrão americano YYYYMMDD, em que YYYY é o ano com quatro posições, MM é o mês com duas posições e DD é o dia com duas posições. Dessa forma, podemos ordenar seus valores e obter o mínimo e o máximo com funções aritméticas simples.

SQL DML: INSERT

(a)

```
INSERT INTO ALUNO (NOME, DATA_NASCIMENTO, SEXO, ENDERECO_CEP, ENDERECO_NUMERO, E_MAIL)
VALUES ('Artur da Nobrega', 20011116, 'Masculino', 20541450, '123',
'tutuzinho111@novoemail.com');
```

(b)

```
INSERT INTO ALUNO
VALUES (NULL, 'Douglas Simões', '443', ' ', 20541450, 19981012, 'Masculino',
'dg_do_443@novoemail.com');
```

(c)

```
INSERT INTO ALUNO (NOME, DATA_NASCIMENTO, SEXO, ENDERECO_CEP, ENDERECO_NUMERO, E_MAIL)
VALUES ('Julia Ortega', 19900706, 'Nenhum', 20541450, '654',
'juju_ort@dominiopublico.com.br');
```

[15:55:09] Erro ao executar consulta SQL no banco de dados 'ead': CHECK constraint failed: SEXO_INVALIDO

(d)

```
INSERT INTO ALUNO
VALUES ('Marcos Damiao', '771', 'bloco B apto 201', 20541450, 19940421, 'Masculino',
'damiao_marcos@oglobo.com');
```

[15:58:52] Erro ao executar consulta SQL no banco de dados 'ead': table ALUNO has 8 columns but 7 values were supplied

Usa-se o comando INSERT para a carga inicial de dados e posterior manutenção do banco de dados com novas informações. Entretanto, como em qualquer sistema, pode ser necessário alterar ou mesmo excluir algum dado, por conta de erros de manuseio ou adequação ao sistema em uso. Como em qualquer linguagem de programação que use arquivos, é possível fazer alterações nos dados por meio de uma exclusão seguida de nova inserção. Entretanto, para grandes volumes em sistemas de bancos de dados, isso pode ser inviável. Assim, a linguagem SQL oferece, além do comando DELETE, que exclui instâncias de dados, o comando UPDATE, que permite alterar dados persistidos, mantendo o controle de integridade em cada operação.

DML: UPDATE

A imagem a seguir traz cinco exemplos de UPDATE utilizando a tabela ALUNO. A sintaxe geral corresponde à da primeira tabela coluna **(3)**. O primeiro caso **(a)** atualiza a coluna ENDERECO_CEP do aluno cuja MATRICULA é igual a 5. A cláusula WHERE especifica qual ou quais tuplas da tabela serão afetadas com a atualização e, nesse caso, em se tratando da MATRICULA, só existe uma tupla, uma vez que essa coluna é a chave primária da tabela. No caso **(b)**, tem-se uma tentativa com falha de atualizar o valor da coluna E_MAIL para o aluno com a mesma matrícula. A atualização falha porque a coluna E_MAIL é obrigatória e, por isso, não é possível especificar o valor NULL (vide mensagem de erro em vermelho). O caso **(c)** também é mais um exemplo com falha. Houve uma tentativa de atualizar a coluna ENDERECO_CEP com um valor que não existe na tabela CEP_CORREIOS e, assim, a restrição de integridade de chave estrangeira seria violada. Infelizmente, a mensagem em vermelho não deixa claro qual *constraint* de chave estrangeira está sendo violada (a tabela ALUNO só tem uma, mas poderia ter mais de uma).

No caso **(d)**, tem-se um exemplo de atualização de duas colunas em um mesmo comando UPDATE. Mas, apesar de os valores violarem restrições das respectivas colunas, nenhuma mensagem de erro é emitida. O que aconteceu, nesse caso, é que nenhuma instância foi encontrada na base de dados para a cláusula WHERE, ou seja, não existe um aluno cujo NOME atenda à condição. Em comandos de UPDATE e DELETE, primeiro as tuplas a serem atualizadas precisam ser localizadas com a cláusula WHERE. O último caso é de um UPDATE sem WHERE **(e)**. Esse caso atualiza TODAS as linhas da tabela ALUNO para a coluna ENDERECO_COMPLEMENTO com valor NULO. Deve-se ter muito cuidado ao utilizar essa opção em produção.

SQL DML: UPDATE

(a)

```
UPDATE ALUNO
SET ENDERECO_CEP = 21990123
WHERE MATRICULA = 5;
```

(b)

```
UPDATE ALUNO
SET E_MAIL = NULL
WHERE MATRICULA = 5;
```

[16:28:07] Erro ao executar consulta SQL no banco de dados 'ead': NOT NULL constraint failed: ALUNO.E_MAIL

(c)

```
UPDATE ALUNO
SET ENDERECO_CEP = 21990555
WHERE NOME = 'Eva Gomes';
```

[16:33:15] Erro ao executar consulta SQL no banco de dados 'ead': FOREIGN KEY constraint failed

(d)

```
UPDATE ALUNO
SET ENDERECO_CEP = 21990555, SEXO = 'Nenhum'
WHERE NOME = 'Julieta Robrigues';
```

(e)

```
UPDATE ALUNO
SET ENDERECO_COMPLEMENTO = NULL;
```

DML: DELETE

A sintaxe geral do comando DELETE corresponde à da primeira tabela coluna **(4)** e a imagem a seguir traz dois exemplos. No exemplo **(a)**, tem-se a remoção de uma linha da tabela ALUNO utilizando como critério o E_MAIL. Como essa coluna não é chave primária nem tem uma restrição de integridade de valor único, é possível que esse comando remova mais de uma linha da tabela, caso existam outras linhas. O caso **(b)** é um DELETE sem WHERE na tabela CORREIOS_CEP. Esse comando falha por restrição de integridade de chave estrangeira, uma vez que já existem na tabela

ALUNO instâncias que apontam para uma linha na tabela
CORREIOS_CEP.



Aprenda mais

Infelizmente, não existe o comando **TRUNCATE TABLE** no SQLite. Mas outros SGBDs oferecem esse recurso, que é equivalente a remover todos os dados das tabelas sem tirar o objeto do esquema do banco de dados. No entanto, com o TRUNCATE, o espaço utilizado pelos dados da tabela é liberado, o que não acontece no DELETE, e o comando, em geral, é executado mais rápido, pois gera um menor volume de *logs* no banco de dados. Para que o espaço liberado seja devolvido ao banco de dados depois de um DELETE sem cláusula WHERE no SQLite, é necessário realizar um DROP TABLE.

Alguns autores consideram o TRUNCATE um comando de DML, pois manipula as tuplas das tabelas, enquanto outros o consideram um comando de DDL, já que algumas informações do dicionário de dados do SGBD também são atualizadas.

Acompanhe os exemplos:

SQL DML: DELETE

(a)
DELETE FROM ALUNO
WHERE E_MAIL = 'jojotody@novoemail.com';

(b)
DELETE FROM CEP_CORREIOS;

[17:13:09] Erro ao executar consulta SQL no banco de dados 'ead': FOREIGN KEY constraint failed

DQL: SELECT

Anteriormente, exercitaram-se as consultas com comandos SELECT no dicionário de dados para verificar os objetos criados. Da mesma forma, pode-se usar o comando SELECT para consultar os dados do banco de dados de uma aplicação, mas, nesse caso, serão usadas as tabelas do esquema criado. Para revisar a sintaxe geral, reveja a figura a seguir:

DQL

SELECT básico	SELECT avançado
(2) SELECT column1, column2, column3 AS c3 ... FROM table1 WHERE condition;	(2) SELECT [DISTINCT ALL] column1, column2, ... FROM table1 [JOIN type JOIN table2 ON condition] WHERE condition GROUP BY column1, column2, ... HAVING condition ORDER BY column1 [ASC DESC], column2 [ASC DESC], ...;

Todos os comandos de INSERT, UPDATE e DELETE podem ser acompanhados de comandos SELECT para verificar o estado do banco de dados antes e depois das operações de manipulação de instâncias. A figura a seguir traz alguns exemplos que podem ser combinados com os exemplos de comandos DML apresentados anteriormente:

SQL DQL: SELECT

(a)
SELECT *
FROM ALUNO;

(b)
SELECT *
FROM CEP_CORREIOS
WHERE CEP = 21990555;

(c)
SELECT *
FROM ALUNO
WHERE NOME = 'Douglas Simões' AND E_MAIL = 'dg_do_443@novoemail.com';

(d)
SELECT *
FROM ALUNO
WHERE ENDERECO_CEP = 20541450 OR ENDERECO_CEP = 21990123;

(e)
SELECT BAIRRO, CIDADE
FROM CEP_CORREIOS
WHERE ESTADO IN ('RJ','SP');

(f)
SELECT ENDERECO_COMPLEMENTO
FROM ALUNO
WHERE ENDERECO_COMPLEMENTO IS NOT NULL;

(g)
SELECT *
FROM ALUNO
WHERE ENDERECO_NUMERO BETWEEN 100 AND 200;

No exemplo (a), tem-se uma consulta que recupera todas as tuplas da tabela ALUNO. O caractere * indica que todas as colunas devem ser recuperadas também. Logo, esse comando mostra todo o conteúdo da tabela, tanto em termos de linhas como de colunas. Na próxima figura, tem-se o estado final da tabela depois que os comandos DML apresentados anteriormente foram executados:

SELECT * FROM ALUNO;

MATRICULA	NOME	ENDERECO_NUMERO	ENDERECO_COMPLEMENTO	ENDERECO_CEP	DATA_NASCIMENTO	SEXO	E_MAIL
1	Antonio dos Santos	15		20541450	19780715	Masculino	antony@meuemail.com
2	Ingrid Belo	35		20541450	19810921	Feminino	ingrid_belo@yahoo.com
3	Eva Gomes	48		20541450	19900104	Feminino	eva1234@click21.com.br
4	Artur da Nobrega	123		20541450	20011116	Masculino	tutuzinho111@novoemail.com
5	Douglas Simões	443		21990123	19981012	Masculino	dg_do_443@novoemail.com

Já no exemplo **(b)**, tem-se novamente o caractere * indicando que todas as colunas devem ser recuperadas, mas esse comando apresenta um filtro por meio da cláusula **WHERE**. Esse filtro recupera somente as tuplas que satisfaçam a condição para o valor do CEP da tabela CORREIOS_CEP. Como essa coluna é chave primária da tabela, se existir alguma tupla, será somente uma. Esse exemplo de consulta é útil em caso de chaves estrangeiras, uma vez que, verificando se o CEP existe antes do INSERT na tabela ALUNO, evita-se o erro de violação de restrição de integridade referencial.

O exemplo **(c)** também apresenta um filtro. Esse filtro recupera somente as tuplas que satisfaçam **ambas** as condições, já que foi utilizado um **AND**. Ou seja, as tuplas recuperadas devem apresentar o valor das colunas NOME e E_MAIL correspondentes a *'Douglas Simões'* e *'dg_do_443@novoemail.com'*, respectivamente. Esse comando pode ser executado antes e depois do respectivo INSERT do aluno com o nome *'Douglas Simões'*. Ao executar antes, verifica-se se a tupla não existe no banco de dados e, ao repetir depois, confirma-se que o comando de INSERT foi executado com sucesso, obtendo-se o número da MATRICULA que foi gerado automaticamente para o aluno.

No caso do SELECT **(d)**, o filtro recupera as tuplas que satisfaçam uma das condições especificadas na cláusula WHERE, uma vez que foi utilizado o operador **OR** – nesse caso, as tuplas correspondentes a cada um dos CEPs listados. Uma sintaxe alternativa para esse filtro, quando a condição está relacionada somente com uma coluna, é a utilização do operador **IN**, como no exemplo **(e)**. As tuplas recuperadas correspondem a qualquer um dos valores listados entre colchetes, ou seja, serão recuperadas da tabela CORREIOS_CEP somente as tuplas correspondentes aos valores da coluna ESTADO especificados. Nesse exemplo, também foram especificadas quais colunas devem ser recuperadas (BAIRRO e CIDADE) na cláusula SELECT.

Quando uma coluna não tem valores, ou seja, seus valores são NULOS, é possível realizar um filtro específico. O exemplo **(f)** apresenta uma consulta que recupera todos os alunos cujo valor da coluna ENDERECO_COMPLEMENTO não seja nulo, e para isso foi usada a sintaxe **IS NOT NULL**. Se a intenção fosse obter o complemento desse resultado, ou seja, recuperar as tuplas em que esse valor não estivesse preenchido, então poderia ser usada a sintaxe **IS NULL**. O exemplo **(g)** apresenta um filtro entre intervalo de valores utilizando a sintaxe **BETWEEN** valor_minimo **AND** valor_maximo.



Fique Ligado

Todos os casos de cláusula WHERE das consultas SELECT podem ser aplicados às cláusulas WHERE de comandos UPDATE e DELETE.

OUTROS RECURSOS DAS CONSULTAS

Em várias situações, é preciso somente obter uma amostra dos dados ou fazer consultas mais específicas sobre seus valores. Nesses casos, dois recursos da linguagem SQL podem ser muito úteis: funções e subconsultas.

Funções

As funções recebem um ou mais valores como parâmetros. Esses valores podem ser colunas das tabelas e retornam um valor

resultante que pode ser recuperado como uma coluna ou usado em uma condição na cláusula WHERE.

Os exemplos **(a)** e **(b)** da figura a seguir trazem casos interessantes. A consulta **(a)** obtém os valores distintos de uma coluna. A função **DISTINCT** seleciona os valores distintos da coluna especificada entre parênteses (nesse caso, SEXO). Já a consulta **(b)** obtém o total de tuplas da tabela ALUNO utilizando a função **COUNT** em relação à chave primária MATRICULA. É importante ressaltar que a coluna escolhida para a contagem não precisa ser a chave primária, mas, se for uma coluna que contém valores NULOS, o resultado da função COUNT não irá corresponder ao número total de tuplas da tabela, mas ao número total de tuplas cujo atributo não é NULO.

Funções SQL

(a)

```
SELECT DISTINCT (SEXO)
FROM ALUNO;
```

(b)

```
SELECT COUNT (MATRICULA)
FROM ALUNO;
```

(c)

```
SELECT MATRICULA, MAX (DATA_NASCIMENTO)
FROM ALUNO;
```

(d)

```
SELECT MATRICULA, MIN (DATA_NASCIMENTO)
FROM ALUNO;
```

(e)

```
SELECT MATRICULA, ENDereco_CEP, ENDereco_NUMERO, COALESCE (ENDereco_COMPLEMENTO,
'SEM COMPLEMENTO') AS COMPLEMENTO_ENDereco
FROM ALUNO;
```

(f)

```
SELECT COUNT (DISTINCT (ESTADO)) AS ESTADOS_QTD
FROM CEP_CORREIOS;
```

(g)

```
SELECT MATRICULA, SUBSTR (CAST (DATA_NASCIMENTO AS TEXT), 1, 4) AS ANO,
SUBSTR (CAST (DATA_NASCIMENTO AS TEXT), 5, 2) AS MES, SUBSTR (CAST (DATA_NASCIMENTO AS
TEXT), 7, 2) AS DIA
FROM ALUNO;
```

O valor máximo da coluna DATA_NASCIMENTO, obtido por meio da consulta **(c)**, utilizando a função **MAX**, permite identificar qual é o aluno mais novo. Já o valor mínimo, exemplificado na consulta **(d)**, por meio da função **MIN**, obtém o aluno mais velho.

A função **COALESCE** é utilizada para manipulação de valores NULOS. Essa função recebe dois parâmetros: a coluna que deve ser avaliada quanto ao conteúdo ser NULO (ausência de valor) e o valor a ser substituído quando um conteúdo NULO for encontrado. No exemplo **(e)**, a coluna ENDereco_COMPLEMENTO pode apresentar valores NULOS, pois não é obrigatória. Nos casos em que isso ocorrer, a ausência de valor será substituída pelo texto “SEM COMPLEMENTO” somente nos dados a serem exibidos. Essa função não afeta o conteúdo já armazenado no banco de dados. Outra característica da sintaxe SQL observada nesse exemplo é o uso de **ALIAS** por meio do **AS**. O *alias* funciona como um apelido para colunas e tabelas a ser

referenciado somente dentro da consulta, ou seja, não afeta o nome da coluna ou da tabela no dicionário de dados.

As funções podem ser usadas de modo aninhado, ou seja, o resultado de uma função pode ser usado como parâmetro para outra função. Há um caso assim no exemplo da consulta **(f)**. Primeiro, são obtidos os valores distintos do ESTADO para a tabela CORREIOS_CEP por meio da função DISTINCT, e o resultado é usado como entrada para a função COUNT, que retorna quantos valores distintos foram encontrados em uma coluna nomeada como ESTADOS_QTD. Essa coluna não existe na tabela ALUNO e é gerada em tempo de consulta como uma coluna derivada.

No último exemplo **(g)**, há outro caso de aninhamento de funções. Primeiro, a função **CAST** converte a coluna DATA_NASCIMENTO, cujo tipo é **NUMERIC**, em **TEXT**. A função CAST recebe o valor de uma coluna de entrada e gera um novo valor convertendo o tipo de dados. Essa conversão é necessária para utilizar a função **SUBSTR**. Essa função recebe um valor do tipo **TEXT** como entrada, a posição inicial e o tamanho, e retorna uma parte desse valor, iniciando na posição informada e com o tamanho desejado. No exemplo, a coluna DATA_NASCIMENTO está sendo convertida em texto e as partes referentes a **ANO**, **MÊS** e **DIA** estão sendo extraídas do texto com base em sua posição inicial e tamanho (lembrando que os valores foram armazenados no formato **YYYYMMDD**).

Funções para TEXTO

Funções de manipulação de texto podem ser encontradas abaixo:

SUBSTR	↓
Extraí e retorna parte do texto com comprimento predefinido, começando em uma posição especificada no texto de origem.	
TRIM	↓
Retorna uma cópia de um texto que contém caracteres especificados removidos do início e do fim do mesmo.	
LTRIM	↓
Retorna uma cópia de um texto que contém caracteres especificados removidos do início do mesmo.	
RTRIM	↓
Retorna uma cópia de um texto que contém caracteres especificados removidos do fim do mesmo.	

As três funções acima podem ser usadas para remover espaços em branco no início e no fim do texto.

LENGTH	↓
Retorna o número de caracteres em um texto ou o número de <i>bytes</i> em um BLOB, ou seja, o tamanho do valor.	
REPLACE	↓

Retorna uma cópia do texto original em que cada ocorrência de uma parte do texto é substituída por outro texto.

UPPER

↓

Retorna uma cópia do texto original em que cada caractere é convertido em letra maiúscula.

LOWER

↓

Retorna uma cópia do texto original em que cada caractere é convertido em letra minúscula.

INSTR

↓

Analisa o texto e retorna um número inteiro abaixo da posição da primeira ocorrência de uma parte de texto especificada.

Operador de concatenação ||

↓

Concatena duas *strings* em uma única *string*.

Operador de concatenação de texto ||

↓

Concatena dois textos, retornando um único texto.

Subqueries

Uma subconsulta (*subquery*), ou consulta interna ou aninhada, é uma consulta dentro de outra consulta SQL incorporada à cláusula WHERE. As subconsultas podem ser usadas com as instruções SELECT, INSERT, UPDATE e DELETE junto com os operadores =, <, >, >=, <=, EXISTS, NOT EXISTS, IN e NOT IN, e devem ser escritas entre parênteses. Essas subconsultas são usadas para retornar dados que serão usados na instrução principal como condição para restringir os dados a serem recuperados.

Uma subconsulta usada como filtro na cláusula WHERE ou atribuição na cláusula SET pode ter apenas uma coluna na cláusula SELECT. As colunas que pertencem à instrução principal podem ser referenciadas na subconsulta ao comparar com suas colunas selecionadas. As subconsultas que retornam mais de uma linha só podem ser usadas com operadores de vários valores, como IN e EXISTS.

Na próxima figura, são apresentadas algumas opções de sintaxe desses comandos. O caso **(1)** mostra a sintaxe geral para SELECT com subconsulta na cláusula WHERE. Esse caso atende à maioria dos operadores, exceto **EXISTS**. O caso **(2)** apresenta a sintaxe específica para esse operador. Observe que não é especificada nenhuma coluna antes do operador, uma vez que a condição EXISTS testa a existência de algum valor recuperado e, se afirmativo, a condição é considerada verdadeira.

Os casos **(3)** e **(4)** trazem a sintaxe geral para o comando de INSERT com subconsultas. A diferença está em que **(4)** especifica quais são as colunas a serem inseridas de acordo com as colunas recuperadas na subconsulta, enquanto **(3)** as insere em todas as colunas da tabela de destino na ordem que estão sendo recuperadas na subconsulta da tabela de origem. Esse comando é particularmente útil quando se deseja fazer uma cópia da tabela antes de um processo de atualização em lote.

Subconsultas também podem ser usadas nos comandos de UPDATE de duas formas: na cláusula **WHERE**, como apresentado no caso (5), e na cláusula **SET**, como apresentado no caso (6). Observe que o caso (5) está aderente à regra referente a ter apenas uma coluna na cláusula SELECT. Com isso, o operador de igualdade compara a coluna1 com o resultado da subconsulta. Além disso, o resultado da subconsulta retorna somente um valor, ou seja, o resultado da função especificado no SELECT. No caso (6), a sintaxe também está aderente à regra referente a ter apenas uma coluna na cláusula SELECT. Com isso, o operador de atribuição irá alterar o valor da coluna1 com o resultado da subconsulta. Do mesmo modo, o resultado da subconsulta retorna somente um valor, ou seja, o resultado da função especificado no SELECT que é usado para a atribuição.

Por fim, o caso (6), para o comando DELETE, permite a subconsulta somente na cláusula WHERE. Ao utilizar o operador NOT IN, a subconsulta pode retornar uma lista de valores, e o valor da coluna1 será verificado nessa lista. As tuplas cujo valor da coluna1 não sejam encontradas na lista serão apagadas.

Subconsultas

Exemplos da sintaxe	
(1) SELECT coluna1, coluna2, ... FROM tabela1 WHERE colunaX OPERADOR (SELECT coluna10, ... FROM tabela2 WHERE condição);	(2) SELECT coluna1, coluna2, ... FROM tabela1 WHERE EXISTS (SELECT coluna10, ... FROM tabela2 WHERE condição);
(3) INSERT INTO tabela_destino (SELECT * FROM tabela_origem WHERE condição);	(4) INSERT INTO tabela_destino (coluna1, coluna2, ...) SELECT coluna10, coluna20, ... FROM tabela_origem WHERE condição;
(5) UPDATE tabela2 SET coluna1 = coluna1 * 1.1 WHERE coluna1 = (SELECT MIN(coluna2) FROM tabela1);	(6) UPDATE tabela2 SET coluna1 = (SELECT MAX(coluna2) FROM tabela1) WHERE coluna2 = valor2;
(6) DELETE FROM tabela2 WHERE coluna1 NOT IN (SELECT coluna2 FROM tabela1 WHERE condição);	

Veja, na próxima figura, quatro exemplos de subconsultas utilizando o esquema da aplicação do EAD. No exemplo (a), a consulta principal irá recuperar todos os alunos cujo ENDERECO_CEP corresponda a algum valor da coluna CEP da tabela CORREIOS_CEP. Como, por construção desse esquema, existe uma restrição de integridade em que somente são inseridos alunos cujo CEP exista na tabela de referência, essa consulta deve recuperar todas as tuplas da tabela ALUNO. No entanto, em um cenário em que essa restrição de integridade não tivesse sido criada ou estivesse desabilitada, o comando retornaria somente os casos em que existisse a correspondência. Nesse cenário, ao trocar o operador de IN para NOT IN, seria possível recuperar as tuplas da tabela ALUNO que apresentam inconsistência com a regra de negócio da aplicação. Essas tuplas deveriam ser ajustadas manualmente antes da criação/habilitação da restrição de integridade.



Fique Ligado

Alguns SGBDs, como o Oracle, permitem que as *constraints* sejam desabilitadas e depois habilitadas novamente. No entanto, essa é uma operação que deve ser feita com muita cautela depois que a aplicação já estiver em produção. Se a *constraint* estiver desabilitada (ou tiver sido removida), dados podem ser inseridos sem estarem aderentes à regra de negócio. Se isso acontecer, a *constraint* só poderá ser habilitada novamente (ou criada) se os dados forem corrigidos.

Suponha que tenha sido feita uma carga de dados na tabela CORREIOS_CEP e o acesso a essa tabela esteja lento em função do volume de dados. Uma opção para reduzir a tabela poderia ser remover as tuplas referentes aos CEPs em que não existam alunos no endereço. Com o exemplo de comando em **(b)**, é possível verificar se a quantidade de tuplas a ser removida com esse critério seria significativa para reduzir o tamanho da tabela. Em caso afirmativo, ou seja, se a decisão for pela remoção, então o comando em **(c)** realizaria esse procedimento de limpeza de dados na tabela. Porém, antes da deleção, pode ser importante realizar uma cópia dos dados para uma tabela auxiliar com a mesma estrutura da tabela CORREIOS_CEP. Com o exemplo do comando em **(d)**, é realizada uma cópia de todos os dados da tabela CORREIOS_CEP para a tabela CEP_COPIA, que foi criada com o mesmo comando CREATE TABLE, trocando somente o nome da tabela.

No exemplo **(e)**, temos o uso do operador NOT EXISTS. Essa consulta permite responder a pergunta: *Quantos cursos não têm alunos inscritos?*. A consulta principal conta o número de cursos que não têm correspondência na subconsulta. A subconsulta acessa a tabela de inscrições e realiza um filtro em que o código do curso da tabela CURSO é igual ao código do curso da tabela Inscrito_Em. Uma vez que o nome da coluna a ser usado na comparação é igual em ambas as tabelas, é necessário identificar de qual tabela está sendo feita a referência. Para isso, cada tabela pode receber um *alias*, que é usado como sufixo na coluna. O *alias* para a tabela CURSO é *c* (em azul), e o *alias* para a tabela Inscrito_Em é *i* (em verde). O *alias* de tabela, assim como o *alias* de colunas, é um apelido válido somente no contexto do comando que está sendo executado. A subconsulta pode referenciar colunas das tabelas da consulta principal, mas o contrário não é possível, ou seja, a consulta principal não pode referenciar colunas das tabelas acessadas somente pela subconsulta.

Subconsultas

(a)
SELECT *
FROM ALUNO
WHERE ENDEREÇO_CEP IN
(SELECT CEP FROM CEP_CORREIOS);

(b)
SELECT COUNT(CEP)
FROM CEP_CORREIOS
WHERE CEP NOT IN
(SELECT ENDEREÇO_CEP FROM ALUNO);

(c)
DELETE FROM CEP_CORREIOS
WHERE CEP NOT IN
(SELECT ENDEREÇO_CEP FROM ALUNO);

(d)
INSERT INTO CEP_COPIA
SELECT * FROM CEP_CORREIOS;

(e)
SELECT COUNT(*)
FROM CURSO AS c
WHERE NOT EXISTS
(SELECT *
FROM Inscrito_Em AS i
WHERE c.CODIGO = i.CODIGO);



OUTROS OBJETOS DO BANCO DE DADOS

Além de tabelas, colunas e restrições, um esquema de banco de dados relacional pode conter outros tipos de objetos. Agora, será apresentado um exemplo simples de três tipos: visões, gatilhos e índices, conforme figura. Esses objetos apresentam variações e configurações mais avançadas que não estão no escopo desse curso.

Objetos

(a)
CREATE VIEW DADOS_PESSOAIS_ALUNOS AS
SELECT MATRICULA, NOME, E_MAIL,
(SUBSTR(CAST(DATA_NASCIMENTO AS TEXT),7,2) || '/' || SUBSTR(CAST(DATA_NASCIMENTO AS
TEXT),5,2) || '/' || SUBSTR(CAST(DATA_NASCIMENTO AS TEXT),1,4)) AS NASCIDO_EM
FROM ALUNO;

SELECT *
FROM DADOS_PESSOAIS_ALUNOS;

(b)
CREATE TRIGGER INTERVALO_INSCRICAO BEFORE UPDATE OF DATA_FINAL_INSCRICAO
ON CURSO FOR EACH ROW
BEGIN
SELECT
CASE
WHEN NEW.DATA_FINAL_INSCRICAO IS NOT NULL AND
NEW.DATA_FINAL_INSCRICAO < NEW.DATA_INICIO_INSCRICAO THEN
RAISE(ABORT, 'Data Final não pode ser anterior a Data Inicio')
END;
END;

UPDATE CURSO
SET DATA_INICIO_INSCRICAO = 20240411, DATA_FINAL_INSCRICAO = 20240401
WHERE CODIGO = 2;
[02:43:17] Erro ao executar consulta SQL no banco de dados 'ead': Data Final não pode ser
anterior a Data Inicio

(c) **CREATE INDEX IDX_CATEGORIA_CURSO ON CURSO (CATEGORIA);**



Visões

As visões são um recurso interessante para que consultas importantes sejam armazenadas no dicionário de dados do banco de dados. Elas funcionam como se fossem tabelas virtuais, uma vez que podem ser referenciadas na cláusula FROM por comandos SELECT.

No entanto, as visões não têm dados armazenados como as tabelas, ou seja, não ocupam espaço no banco de dados, e os dados recuperados ao acessar uma visão são os das tabelas a que ela faz referência.

Uma visão pode fazer referência a mais de uma tabela. A criação de uma visão é bem simples: **CREATE VIEW** nome_da_visão **AS** (subconsulta).

Na figura anterior, tem-se um exemplo **(a)** de criação de uma visão com quatro colunas da tabela ALUNO. Nesse caso, é realizado um tratamento para a formatação da data de nascimento, que envolve: converter o número em *strings* (CAST); obter partes das *strings* (SUBSTR); e concatenar (||) em uma única *string*,no formato DD/MM/YYYY. Se esse tratamento for recorrente na aplicação, a criação da visão pode ser um recurso para evitar repetir esse código em todas as consultas, ou seja, a visão promove o reuso de parte do código das consultas. O comando de consulta a uma visão é igual ao de consulta a uma tabela (como ilustrado nesse exemplo também).



Aprenda mais

No SQLite, as visões permitem somente consultas, mas em outros SGBDs é possível realizar operações de INSERT, UPDATE e DELETE nas tabelas originais usando visões.

Gatilhos

Gatilhos são funções executadas automaticamente para cada tupla quando ocorre um evento de banco de dados especificado. Os eventos podem ser comandos de DELETE, INSERT ou UPDATE de uma tabela, ou um UPDATE em uma ou mais colunas especificadas de uma tabela. Vale ressaltar que o comando TRUNCATE não aciona gatilhos de DELETE. O código da função do gatilho pode fazer referência aos valores que estão armazenados na tabela ou aos novos das colunas usando os sufixos OLD e NEW. Os gatilhos podem ser acionados antes ou depois de executar o comando por meio das cláusulas BEFORE ou AFTER. Uma função SQL especial, RAISE(), pode ser usada dentro do código da função de gatilho para gerar uma exceção e, com isso, evitar que o comando seja executado. A estrutura geral do comando de criação de uma *trigger* é:

```
CREATE TRIGGER nome_do_gatilho [BEFORE | AFTER] [UPDATE OF column_name | INSERT | UPDATE | DELETE] ON tabela
BEGIN
  (Código da trigger)
END;
```

Suponha que a seguinte regra de negócio seja identificada: a data final da inscrição de um curso não pode ser menor que a data inicial. Essa regra pode ser checada na aplicação, mas, se for necessário que essa verificação seja feita no banco de dados, o gatilho pode ser um recurso. Na figura anterior a esta última, em **(b)**, é apresentado um exemplo para a implementação dessa regra por meio de um gatilho chamado INTERVALO_INSCRICAO. Esse gatilho será executado

ANTES da atualização da coluna DATA_FINAL_INSCRICAO da tabela CURSO. Dentro do código da função, é realizada a comparação entre o novo valor da DATA_FINAL_INSCRICAO e o da DATA_INICIO_INSCRICAO. Em caso de violação da regra de negócio, uma exceção é gerada com a função RAISE, sendo o comando de UPDATE cancelado e retornada uma mensagem de erro. O exemplo de comando de UPDATE ilustrado a seguir irá falhar, pois não atende à regra de negócio.

Índices

Os índices são estruturas de dados auxiliares que o mecanismo de pesquisa do banco de dados pode usar para acelerar a recuperação de dados.

Os índices podem ser criados ou eliminados sem nenhum efeito no conteúdo dos dados das tabelas, mas, diferentemente das visões, ocupam espaço no banco de dados.

Assim sendo, devem ser criados com critério e submetidos a posterior acompanhamento para avaliar sua efetividade de uso. Em termos gerais, não é recomendável criar índices para:

- tabelas com poucas tuplas;
- tabelas com operações frequentes, como grandes atualizações ou inserção em lote;
- colunas que contêm um grande número de valores NULL;
- colunas que são frequentemente alteradas.

A criação de um índice envolve a instrução **CREATE INDEX** nome_indice **ON** tabela (coluna1, coluna2, ...), que permite nomear o índice e especificar a tabela e as colunas a serem indexadas. No exemplo (c) da figura *Objetos* acima, é apresentada a criação de um índice de pesquisa na tabela CURSO para a coluna CATEGORIA. Esse índice pode ser útil, caso a aplicação faça filtros de curso por categoria quando o aluno está buscando em qual curso deseja se inscrever e a tabela de cursos tenha muitas tuplas.



Fique Ligado

Atualizar uma tabela com índices leva mais tempo do que atualizar uma tabela sem (porque os índices também precisam de atualização). Portanto, crie índices apenas em colunas que serão pesquisadas com frequência.

Um índice ajuda a acelerar consultas, subconsultas e cláusulas WHERE, mas retarda a entrada de dados, com instruções UPDATE e INSERT.

IMPORTAÇÃO DE DADOS EM LOTE

É comum que quem trabalhe com bancos de dados precise realizar a importação de dados de arquivos externos, bem como a exportação de dados para arquivos externos. Em casos de grande volume, existe a opção de carregar os dados em lote, ou seja, realizar uma operação

conhecida como BULK INSERT em SQL. A sintaxe dessa operação pode variar entre SGBDs. Em outros casos, essas operações podem ser realizadas utilizando interfaces de ferramentas de extração, transformação e carga (ETL), e a sintaxe do comando acaba sendo irrelevante. **O importante é saber que existe essa opção e usá-la para realizar o processo de carga em lote de modo otimizado.**

Carga em lote por tabela

No SQLite, a carga em lote por tabela pode ser executada da seguinte forma:

Abra o `prompt` de comando e navegue até a pasta `c:\sqlite`:

```
cd c:\sqlite
```

↓

Execute o comando de criação do banco de dados:

```
sqlite3 EAD.db
```

↓

Execute o comando de criação da tabela-destino no banco de dados:

```
create table tabela_destino (coluna1, coluna2, coluna3...);
```

↓

Especifique o `separador` dos valores das colunas que foi usado no arquivo:

```
.separator 'caractere_separador'
```

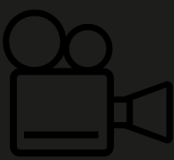
↓

Realize a importação dos dados para a tabela-destino:

```
.import caminho_arquivo\arquivo tabela_destino
```

De modo semelhante, podemos usar a opção de exportação de dados de um banco de dados para arquivos. Esses arquivos podem ser usados para carregar esses dados em outros bancos de dados, sendo uma opção para transferência de dados, inclusive entre ambientes de homologação e produção.

No vídeo a seguir, será apresentada a carga de dados de CEP em uma tabela auxiliar que pode ser posteriormente usada pela aplicação para validar a entrada de dados ou copiar os dados para a tabela que pertence ao esquema da aplicação EAD (CEP_CORREIOS). Além disso, será mostrada a exportação de dados de uma tabela em dois diferentes modos.



Tutorial para carga e exportação de dados no SQLite

Acompanhe no vídeo o passo a passo para carga e exportação de dados utilizando o SQLite.



Ao longo das aulas desta disciplina, você foi apresentado aos conceitos básicos de bancos de dados, com ênfase na teoria e prática. Esses conceitos introdutórios são essenciais para o desenvolvimento de sistemas de bancos de dados. Abordamos a modelagem de dados como resultado da análise de requisitos, incluindo a geração de um DER de aplicações realistas. Depois, focamos parte da linguagem SQL que permite criar objetos no banco de dados (DDL), atualizar instâncias de dados (DML) e recuperar seu conteúdo (DQL). Além disso, também exercitamos a carga e exportação de dados.



Reflita

Após a criação do banco de dados e a implantação da aplicação em produção, os dados reais começam a ser carregados e a aplicação começa a agregar valor à organização. Muitos sistemas são essenciais para a organização e requerem a disponibilidade de funcionamento 24 x 7.

No entanto, isso não impede que novos requisitos sejam identificados ou que alguns erros de modelagem ou identificação de requisitos sejam detectados e precisem ser corrigidos.

Qual é o impacto das alterações no esquema do banco de dados nas aplicações em produção? Como lidar com esses impactos? O que pode ser feito para reduzir o impacto?

Não se esqueça dos pontos vistos na seção “Que história é essa?”, pois serão de extrema importância para a construção do conhecimento desta aula.